Fox Engine y Arquitectura Mother Base

16/3/2026

—

Marcos Bolívar Muñoz

Sebastian Carrillo Medina

Mario Romera Braojos

Francisco Castillo Martín

Javier Perez Martín

Índice

Contexto. **2**
Fox Hound: el equipo **3**
Finalidad del proyecto. **4**
Factor Show: "Ingeniería del Caos" en la presentación 4
Infraestructura. 4
La Arquitectura del Proyecto (Lo que tenéis que levantar en Proxmox) 4

Contexto.

La idea de Fox Engine es crear un cerebro de lo que sería un entorno real de un entorno empresarial pero impulsados al infinito con una IA.

Fox Engine es un ejecutor remoto, tu señalas y el ejecuta todo remotamente desde tu dispositivo móvil mediante la aplicación de mensajería Telegram ya que esta tiene opciones para crear bots que generan una API y puedes integrar de manera gratuita en tu servidor mediante Python.

Ahora bien, este TFG ofrece a su vez la configuración de lo que sería un entorno empresarial real, no solo nos limitaremos a la IA.

Tendremos 4 servidores como mínimo que son:

- Fox Engine:

  Este servidor solo y únicamente aloja a la IA.

  Este entorno ha sido previamente desarrollado por Marcos Bolívar pero el encargado de mejorar radicalmente el entorno será Mario.

- Mother Base:

  Este servidor hará de bastión, será el encargado de vigilar todos los demas servidores y sera el que haga de intermediario entre IA y servidores externos, además de ellos este será el que tendrá Docker Swarm configurado como maestro, es nuestro director de orquesta de todo el cluster Docker.

  Además, tendrá instalado el combo Prometheus/Grafana para la monitorización, si se excede algunos límites la IA actuará en consecuencia.

  Mother Base será desarrollado principalmente por nuestro compañero Marcos Bolívar y contará con el soporte de Sebastián Carrillo en caso de ser necesario.

- Plant:

  Plant es un servidor con una función indeterminada, la intencionalidad es que ofrezca cualquier tipo de servicio que le mandemos desde la IA o cualquier función previamente desarrollada por nuestro compañero Sebastián Carrillo.

- Tanker:

  Tanker tiene exactamente la misma intencionalidad que Plant, ambos compartiran servicios y tendrán su respectivo balanceo de carga pero contará con la libertad total de creación de nuestro compañero Javier.

- En el apartado de seguridad de red haremos uso de de PfSense el cual será configurado íntegramente por Francisco, que se encargará de aislar la red y procurar un entorno seguro para nuestro entorno real.

Fox Hound: el equipo

El equipo formado ha sido designado como la unidad Fox Hound en la que encontramos:

- **Marcos Bolívar Muñoz:**

  Encargado de organizar el proyecto y el equipo; la creación de Mother Base y, junto a Mario, la concepción del servidor Fox Engine.

- **Sebastian Carrillo:**

  Encargado de Plant, tiene libertad total para implementar los servicios que considere oportunos en el servidor, también desempeña una labor de soporte junto a Marcos Bolívar en Mother Base.

- **Javier Perez Martín:**

  Encargado de Tanker, tiene libertad total para implementar los servicios que considere oportunos en el servidor, también desempeña una labor de soporte junto a Francisco para administrar la seguridad.

- **Mario Romera Braojos:**

  Su principal función es la investigación y desarrollo de la IA junto a Marcos, existe una IA actualmente que se encuentra en un estado prematuro. La misión de Mario es mejorar completamente esta IA, darle más funcionalidad y, en controlar que puede hacer y que no debe hacer.

- **Francisco Castillo Martín:**

  Principal responsable de la seguridad de absolutamente todos los servidores y servicios que ofrece nuestra arquitectura.

Finalidad del proyecto.

Construir un entorno cliente-servidor seguro, con funcionalidades modernas usando aplicaciones como Ansible, Docker, Docker Swarm, Prometheus, Graphana, y otras tecnologías basadas en entornos de programación en Python.

Además de esto uno de nuestros servidores es una IA que se encarga de la gestión siempre supeditada por un miembro del equipo Fox Hound que, le dará órdenes a través de Telegram desde cualquier parte del mundo.

Como colofón final provocaremos un desastre para hacer una demostración de lo que puede hacer la IA y como puedo no solo prevenir estos casos si no arreglarlos.

Factor Show: "Ingeniería del Caos" en la presentación.

Montaremos un escenario de "Chaos Engineering" (Ingeniería del Caos).

- La chicha: En la presentación, enseñáis vuestra arquitectura funcionando. Entonces, delante de los profesores, ejecutáis un script "destructor" (podéis usar herramientas como Chaos Mesh o simplemente scripts vuestros) que empiece a saturar la red simulando un ataque DDoS, o a llenar los discos duros virtuales al 100%.
- En la pantalla gigante se verá cómo Grafana se pone en rojo, y vuestro Bot de IA empieza a escupir diagnósticos en el chat: "Ataque de denegación de servicio detectado desde la IP X.X.X.X. Procedo a meter regla en el Firewall de iptables/pfSense". Y la gráfica vuelve a la normalidad en directo.

Infraestructura.

La Arquitectura del Proyecto (Lo que tenéis que levantar en Proxmox)

- **1. El Guardián (Firewall/Router):**
  - **SO:** pfSense o OPNsense.
  - **Recursos:** 1 vCPU, 1GB RAM.
  - **Función:** Le ponemos dos tarjetas de red virtuales en Proxmox. Una conectada a la red del instituto (WAN) y otra a una red interna aislada (LAN). Todo nuestro proyecto irá detrás de este firewall. Así podréis simular ataques y bloqueos de IPs de forma realista sin cargaros la red del centro.

- **2. Nodo Mother Base (El Jefe del Clúster):**
  - **SO:** Debian 12 / Ubuntu Server.
  - **Recursos:** 2 vCPUs, 4GB RAM, 30GB Disco.
  - **Función:** Aquí instalaremos Docker (y activaremos Docker Swarm como mánager). También alojará el cerebro de la monitorización (Prometheus y Grafana).
- **3. Nodos Worker Plant y Tanker (Los Currantes):**
  - **SO:** Debian 12 / Ubuntu Server.
  - **Recursos:** 2 vCPUs, 2GB RAM, 20GB Disco (cada uno).
  - **Función:** Se unirán al clúster de Docker. Aquí es donde se levantarán automáticamente las webs de prueba (por ejemplo, Nginx o WordPress) que la IA vigilará y que nosotros intentaremos "tirar" el día de la presentación.
- **4. Servidor Fox Engine (El Bot de IA):**
  - **SO:** Debian 12 / Ubuntu Server.
  - **Recursos:** 2 vCPUs, 2GB RAM, 20GB Disco.
  - **Función:** Aquí vivirá vuestro bot de Python conectado a Telegram/Discord y a la API de la IA. Tendrá SSH y Ansible instalados para poder mandar órdenes a los demás servidores.

Todo esto estará orquestado en el Proxmox del instituto.

Documento aún en desarrollo...

Entrega 2:

Implementaciones realizadas en nuestro proyecto.

Infraestructura de Red y Gestión de Identidades.

Como responsable de la arquitectura de sistemas, he diseñado y desplegado el ecosistema de red sobre el que operan los microservicios de Fox Engine. Su trabajo constituye el "suelo" técnico del proyecto, garantizando que la comunicación entre nodos sea segura, aislada y que el acceso administrativo sea infalible mediante protocolos criptográficos.

El Guardián: Perímetro de Red con pfSense

Se ha implementado un firewall **pfSense CE** que actúa como Gateway (Puerta de enlace) y centro logístico de la red. Esta configuración es vital para el éxito del TFG por tres motivos:

- **Segmentación y Aislamiento:** El proyecto vive en una red LAN privada (192.168.1.0/24) totalmente aislada. Esto permite ejecutar pruebas de "Ingeniería del Caos" y simular ataques de red de forma realista sin comprometer la infraestructura del centro educativo.
- **Acceso Remoto Seguro (OpenVPN):** Se ha configurado un túnel VPN cifrado bajo estándar SSL/TLS. Esto permite que el equipo (Marcos, Javi, Sebas, Mario) pueda conectarse a la infraestructura interna desde cualquier red externa (hogar o móvil) con total seguridad.
- **Gestión de Tráfico:** El firewall controla cada paquete que entra y sale de los nodos, permitiendo a la IA monitorizar intentos de intrusión en tiempo real.

Mother Base: El Cerebro de Identidades (LDAP)

He dotado a la Mother Base de un **Servicio de Directorio Centralizado (OpenLDAP)**, transformándolo en la autoridad de certificación y confianza del clúster:

- **IAM (Identity and Access Management):** En lugar de gestionar usuarios locales en cada servidor, todas las credenciales del equipo residen en una base de datos centralizada bajo el dominio dc=tfg,dc=local.
- **Administración Web:** Mediante la integración de phpLDAPadmin, se gestionan las Unidades Organizativas de usuarios y grupos, facilitando la escalabilidad del proyecto ante la incorporación de nuevos nodos o colaboradores.

Criptografía y Acceso "Zero-Password"

Para cumplir con el requisito de "ejecución instantánea" de Fox Engine, he implementado un sistema de **Autenticación por Claves Asimétricas (ED25519)**:

- **Eliminación de contraseñas:** Se ha configurado el servicio SSH en todos los nodos (Tanker, Plant, Mother Base) para que confíen en llaves maestras digitales. Esto permite que la IA ejecute órdenes de administración en milisegundos sin pausas por peticiones de credenciales.
- **Persistencia de Usuario:** Mediante el uso de librerías libnss-ldapd y el módulo pam_mkhomedir, el sistema reconoce automáticamente a cualquier usuario del equipo en cualquier servidor, creando su espacio de trabajo (/home/usuario) de forma dinámica al primer inicio de sesión.

Implementación Técnica del Cerebro: Fox Engine

Arquitectura de Inferencia Local: Evolución a Ollama

En la fase actual del proyecto, se ha realizado una migración crítica desde servicios de IA basados en la nube (Google Gemini API) hacia una infraestructura de inferencia local utilizando **Ollama**.

Esta decisión técnica responde a tres pilares fundamentales en entornos empresariales:

- **Privacidad de Datos:** Al procesar las peticiones en el propio servidor, se garantiza que la topología de red, credenciales y scripts sensibles no sean enviados a servidores de terceros.
- **Soberanía Tecnológica:** Se elimina la dependencia de cuotas de tokens, latencias de red externa y costes variables de APIs comerciales.
- **Especialización del Modelo:** Se ha integrado el modelo **qwen2.5-coder,** seleccionado específicamente por su alto rendimiento en la generación de scripts de Bash y administración de sistemas Linux.

Desarrollo del Orquestador Multihilo (Python Backend)

El núcleo de Fox Engine se ha desarrollado en Python, empleando una arquitectura **multithreading** para gestionar dos flujos de trabajo simultáneos:

Servidor de Alertas (Flask): Se ha implementado un microservicio web que escucha de forma pasiva en el puerto 5000. Este componente está diseñado para recibir webhooks de sistemas de monitoreo como **Prometheus y Alertmanager**, permitiendo al motor reaccionar ante eventos críticos del sistema de forma proactiva.

Gestor de Mensajería (Telebot): Un hilo independiente gestiona la comunicación cifrada con la API de Telegram, procesando tanto consultas de lenguaje natural como comandos de administración.

Protocolos de Seguridad y Control de Acceso

Dada la naturaleza crítica de ejecutar comandos remotos, se han programado tres capas de seguridad robustas:

Filtrado por Identificador Único (Whitelist)

El sistema implementa la función es_usuario_autorizado(), que actúa como un firewall de identidad. Cada interacción es validada contra un CHAT_ID específico. Cualquier intento de acceso desde una cuenta no autorizada es bloqueado y registrado en el servidor, impidiendo que terceros puedan interactuar con el bot incluso si descubren su existencia.

Mecanismo de Validación "Human-in-the-Loop"

Para prevenir ejecuciones accidentales o alucinaciones de la IA, se ha implementado un sistema de **Confirmación de Acción**. Cuando la IA genera un comando técnico, el bot no lo ejecuta inmediatamente. En su lugar:

- En primer lugar se presenta al usuario el comando propuesto.
- El usuario debe pulsar físicamente un botón de "✅ EJECUTAR" o "❌ CANCELAR" en su dispositivo móvil.
- Solo tras la validación humana, el comando se envía al shell del sistema.

Lógica de operación y autocuración.

Fox Engine distingue entre la asistencia consultiva y la ejecución técnica mediante el uso de prefijos y prompts de sistema:

- **Modo Consultivo:** El bot actúa como un administrador de sistemas experto, ofreciendo respuestas técnicas breves procesadas bajo el contexto CONTEXTO_GRAL.
- **Modo Ejecutor (/haz):** Al utilizar este comando, el sistema activa un prompt estricto que obliga a la IA a devolver exclusivamente código Bash. La salida es procesada por una función de sanitización (clean_command) que elimina etiquetas Markdown para asegurar que el comando sea procesable por el sistema operativo.
- **Capacidad de Autocuración (Self-healing):** Ante una alerta de Flask (ej. una instancia caída), el sistema consulta automáticamente a Ollama una posible solución técnica. Fox Engine propone al administrador el comando de reparación sugerido, facilitando una respuesta inmediata ante incidentes desde el móvil.

Interacción con el Sistema Operativo.

La ejecución técnica se realiza mediante el módulo **subprocess.run**, configurado con **shell=True** y captura de salida completa (**capture_output**). Esto permite que el administrador reciba en su chat de Telegram el resultado real de la terminal (stdout) o, en su defecto, el seguimiento de errores (stderr), garantizando una transparencia total sobre las acciones realizadas en el servidor remoto.

Para ver el código de la IA iremos al siguiente enlace.

Mother Base: Control total de todo.

Como dejamos claro desde el principio Mother base es un centro neurálgico y le hemos instalado Docker Swarn y Prometheus/Grafana para tener el control de todos los demás servidores y servicios.

El Radar.

Vamos a levantar Grafana y Prometheus. Crea un directorio llamado monitor y entra en él.

nano prometheus.yml

Escribimos solo lo esencial:

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'swarm'
    dns_sd_configs:
      - names: ['tasks.node-exporter']
        type: 'A'
        port: 9100
```

Crea el archivo del clúster:

nano docker-compose.yml

```
services:
  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml"]
    deploy:
      placement:
        constraints: [node.role==manager]

  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
    deploy:
      placement:
```

```
    constraints: [node.role==manager]

  node-exporter:
    image: prom/node-exporter
    hostname: '{{.Node.Hostname}}'
    deploy:
      mode: global
```

Despliega el radar:

docker stack deploy -c docker-compose.yml monitor

Todo esto hace que podamos ver las estadísticas de las máquinas.

Máquina Plant.

Principalmente en este nodo se instalarán diversos servicios en los que los administradores de IT de la empresa podrán apoyarse ya sea levantando contenedores para uso de páginas webs gestionar tickets de soporte de mantenimiento, tener acceso a un gestionador de contraseñas a mano entre otras funcionalidades que puedan tener

Passbolt: Gestionador de Contraseñas

Passbolt es un gesto de contraseñas de código abierto creado para equipos técnicos y organizaciones que requieren de un alto control y seguridad de sus contraseñas. De los rasgos más característicos de Passbolt es su uso de OpenPGP que sirve para dar un modelo de seguridad.

Fundamento Técnico y modelo de seguridad

- Cifrado de extremo a extremo: Passbolt se fundamenta en su modelo de seguridad empleando OpenPGP para dar una confidencialidad e integridad de datos absoluta. Este estándar usa algoritmos criptográficos reconocidos y auditados.
- Control de Clave Privadas: Las claves privadas sólo se generan y guardan en el cliente de passbolt lo que hace que el servidor nunca tenga acceso a ellas en texto plano
- Intercambio Seguro de Claves: El servidor solo guarda las claves públicas y secretos cifrados del usuarios y para que este comparta una contraseña tiene que descifrar localmente y vuelve a cifrar la clave pública del destinatario, enviando el nuevo texto cifrado al servidor

Arquitectura y Componentes

Passbolt usa la arquitectura MVC, aunque la presenta de forma en la que cada componente tiene un rol específico en el flujo de gestión de contraseñas. En el centro está su servidor principal que está hecho en PHP con CakePHP que tienen la lógica de negocio, controla la autenticación, gestiona los permisos y expone una API REST que sirve de puente para los clientes. Y para el uso de los usuarios lo hacen mediante navegadores junto la extensión del propio Passbolt que integra el motor criptográfico OpenPGP. Y todo esto está apoyado en un servidor de base de datos, donde se guardan los metadatos y los secretos cifrados.

FreeScout: Sistema de Atencion al cliente(Helpdesk)

FreeScout es un sistema de helpdesk y bandeja de entrada compartida de código abierto,

presentado como una alternativa ligera a otras soluciones.

Arquitectura y Fundamento Tecnico

FreeScout es una aplicación web basada en PHP, que está hecha sobre un framework Laravel 5.5 haciendo así que tenga una base sólida, bien estructurada y extensible

Componentes

FreeScout tiene un núcleo básico muy simple. Esto significa que solo incluye lo que realmente se

necesita para que funcione como un helpdesk. No es nada complicado. Lo bueno es que se puede ampliar

con módulos extras según lo que cada organización necesite. Estos módulos son gratuitos y de código abierto. Se pueden descargar directamente en el servidor.

Con estos podemos agregar funciones avanzada y conectarlas contras herramientas. Se conecta a los buzones de correo electrónico mediante IMAP y SMTP. Cuando se reciben mensajes, se convierten automáticamente en tickets. Las respuestas que se envían desde la plataforma llegan como correos electrónicos. Esto hace que sea compatible con Gmail, Office 365 o servidores propios. FreeScout también es compatible con herramientas externas mediante una API RESTful. Para ello hay que activar el módulo“API & Webhooks".

Script de Backup Servicios

En el script lo que buscamos es poder hacer copias de seguridad diarias (incrementales)

replicando solo cambios de los directorios críticos del servidor principal hacia un servidor respaldo

con RAID 1. Cada semana se genera una copia completado que será comprimida y cifrada

guardando así toda la información del disco. Y para mayor seguridad estos datos se guardarán en la nube de AWS S3

Nodo Tanker: Gestión y Documentación Centralizada.

Definición del Rol de Tanker en el Ecosistema.

El nodo Tanker ha sido designado para especializarse como el frontend estratégico del clúster. Mientras que otros nodos como Mother Base se encargan de la orquestación y monitorización, y Fox Engine aloja el cerebro de IA, la arquitectura de Tanker está diseñada para facilitar el desarrollo y mantenimiento tanto a corto como a largo plazo, resolviendo la necesidad de persistencia en la documentación técnica y crear un portal de acceso unificado a las herramientas que se van a ir implementando.

Capa de Enrutamiento: Nginx como Proxy Inverso.

En lugar de exponer múltiples puertos hacia pfSense, centralizamos todo el tráfico entrante en un contenedor Nginx. Este actuará como el único punto de entrada para los servicios de gestión del nodo.

- **Abstracción de Puertos:** En lugar de que los administradores tengan que recordar puertos específicos para cada servicio (como el 3000 o el 8080), Nginx permite el acceso mediante nombres de dominio locales (ej. docs.engine.fox).
- **Seguridad y Aislamiento:** Al trabajar dentro de una red interna de Docker, los demás contenedores de Tanker no exponen sus puertos al sistema operativo base. Nginx actúa como un intermediario que recoge la petición en el puerto 80/443 y la redirige de forma segura al contenedor correspondiente.

- **Optimización de Recursos:** Nginx destaca por su baja huella de memoria, algo crítico dado que el nodo Tanker opera con 2 vCPUs y 2GB de RAM. Esto permite que el servidor gestione múltiples peticiones simultáneas sin penalizar el rendimiento del resto de la infraestructura.

Gestión del Conocimiento: MkDocs.

Un proyecto tan extenso y completo requiere una documentación que esté a la altura. Para ello, hemos seleccionado MkDocs, habilitando una opción sencilla de mantener y acceder a información relevante.

- **Flujo de Trabajo Colaborativo:** Al utilizar archivos en formato markdown, cualquier miembro del equipo puede actualizar su sección correspondiente de forma sencilla. Estos archivos de texto plano son posteriormente procesados por el contenedor de MkDocs para generar una web estática profesional.
- **Centralización de Protocolos:** En este portal se documentan desde las reglas de firewall del pfSense hasta los prompts de sistema del modelo qwen2.5-coder. El objetivo es que, ante un fallo en el sistema, cualquier administrador pueda acceder a Tanker y encontrar el manual de mitigación de errores de forma instantánea.
- **Buscador Integrado:** MkDocs genera un índice de búsqueda en tiempo real, permitiendo localizar rápidamente configuraciones de OpenLDAP o parámetros del servidor sin tener que navegar manualmente por carpetas de documentos.